

ELEG5491 Project: Object Momentum Classification Using Semantic Segmentation Deep Neural Networks

Kuiyi Gao

2025.5

Abstract and Motivation

Recently, embodied artificial intelligence has gained massive attention. Humanoid robot companies have thrived in the market and have shown remarkable business potential. According to the Peking University alignment lab, when robotic systems are deployed in the real world, the vision-language-action model can generate harmful content that causes physical harm to humans and the environment[6], typically collisions. These malfunctions should raise high awareness among researchers because robots including self-driving cars, humanoid robots and wheeled robots are autonomous, and they also have a high momentum energy that could be destructive during collisions, according to a CVPR2024 study[5]. In this project, we propose a fast deep neural network that can recognize and map objects with their corresponding movability (instability) level within an image using precise and straightforward criteria, thus providing a precaution for a robotic system to predict movements in advance. The model we trained achieved a highest MIOU of 0.6381 within 50 epochs and a loss of 0.75 on the validation set. The inference time is 58.21 FPS on average of 1000 samples. The computation can be efficiently deployed on an edge computation device such as the NVIDIA Jetson Nano toolkit. This advanced network enhances robots' understanding of the physical world and is provisional in industrial applications.

Dataset

In this section, we will briefly introduce the dataset of our project. We utilize the COCO-Stuff dataset[1], which is a pixel-level annotated object detection and segmentation dataset consists of 164K images selected from COCO2, each paired with a detailed annotation of 182 types of objects. COCO-Stuff also consists of a validation set with 500MiB of image data, on which we can perform the semantic segmentation tasks and evaluate the model's performance. We propose the novel COCO-Stuff-Momentum dataset to facilitate the model in conducting the classification of momentum levels. COCO-Stuff-Momentum is designed according to a series of mapping rules. These rules are based on the real-world understanding of object momentum derived from the knowledge of human annotators and Large Vision Language Models like GPT4o[3]. We map the object with its momentum level using a tag size of 4, each denoting a state of momentum in real world scenarios. The objects with higher momentum is allocated with a tag of higher value according to the detailed rules in Appendix A.

Mapping Rules			
Momentum Level	Description	Original Labels	New Label
Highly Mobile	People, Vehicles and Animals	1-26(roughly)	3
Inactively Mobile	Clothes, Foods and other small objects	27-59(roughly)	2
Normally Stable	Furniture, Appliances and Street Facilities	60-90(roughly)	1
Not Movable	Buildings, Landscape and Plantation	90-182(roughly)	0

Table 1: Table of Mapping Rules from COCO-Stuff to COCO-Stuff-Movability

We store the mapping result in a 2*256 Numpy Array document using a LUT configuration, with a pair of original label and new label in each slot. Apart from that, we defined the output masks produced by our model to highlight objects with higher momentum with [0, 40, 80, 120] grayscale mapping. The mapping process is cross-checked by human annotators and vision language models.

Model Structure

In this section, we will explain the detail of our design of DNN.

- We use the DeepLabV3 structure[2] as the DNN backbone in this project. In practice, we utilize the DeepLabV3-Resnet-50(pretrained) in PyTorch. Compared to DeepLabV3-Resnet-101, the network is relatively lightweight and takes up less computational resources.
- The major modification of our network is that we redesigned the ASPP module in the original network and removed the Image Pooling layer directly after the Atrous Spatial Pyramid Pooling layers of stride [6, 12, 18] . Removing the Image Pooling layer and Atrous Spatial Pyramid Pooling layer of stride 24 in the ASPPHead is estimated to result in a 15%-20% reduction in training cost. This approach can maintain the model’s ability to detect segments within an image.
- We also didn’t adopt the decoder structure of deeplabv3+[4] for faster inference. We adjust the bilinear interpolation layer to a scale of 8, which saves the cost of training as well. In the multi-scale classifier, we restricted the out_channels to be 4, which corresponds to the number of classification tags. The output mask consists of 4 different layers of prediction.

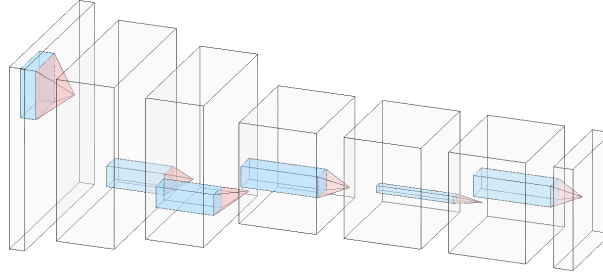


Figure 1: A Diagram of The Proposed Network(Simplifying the ASPPHead)

Loss Function

In this section, we will introduce our new loss function, designed to meet the training goal of the DNN.

- During the training, we apply the weighted cross-entropy loss function to handle backward inputs, defined as:

$$L_{wce}(p, y) = - \sum_{c=0}^C w_c y_c \log(p_c)$$

$$c = 0, 1, 2, 3$$

The weight is designed to enhance the learning of highly-movable objects. Within the dataset, they usually occupy fewer pixels and have smaller quantity. Yet, highly-movable objects are our focus in this research question. We use pixel-level counting to decide the total area(number) of a certain genre and we apply the median frequency balance approach to normalize the weight parameter.

- We denote:

T_c = Pixels in a Dataset Where Tag is c

$$w_c = \frac{\text{median}(\text{freq}(T))}{\text{freq}(T_c) + \epsilon}$$

- We apply the Weighted Soft IoU score to optimize the performance in segmentation and edge-detection, we pass the weight w_c in weighted entropy loss to the IoU calculation:

$$L_{iou} = 1 - \frac{\sum_{c \in P} w_c \text{IoU}_c}{\sum_{c \in P} w_c} = 1 - \sum_{c \in P} w_c \text{IoU}_c$$

$$P = \{c \mid \sum_{i,j} y_{c,i,j} > 0\}$$

- We adopt the two scores stated above into L , here we denote α, β as parameters between $[0, 1]$ to scale the two parameters. initially, we set $\alpha = 1$ and $\beta = 0.5$ to achieve a balance in the convergence of IoU score as well as the loss function.

$$L = \alpha L_{wce} + \beta L_{iou}$$

Training

In this section, we train the model. For the GPU in the context, we rented a third party GPU for the training: 1 * Nvidia GeForce RTX3090(24GB). We store the training configuration in train.py, and store the New DeepLab model description with modified ASPP head in model.py, and store the mapping rules in create_lut.py. Table 2 have provided a detailed file structure of the project.

MomentumDetection		
dataset	COCO-Stuff_train	image
		annotation
	COCO-Stuff_val	image
		annotation
	lut.npy	
src	dataset.py	dataset config.
	train.py	training spec.
	model.py	model structure
	create_lut.py	mapping

Table 2: File Structure of Momentum Detection

To validate our training process, we firstly trained the model using 3200 images with a batch size of 16, learning rate $1e-4$ 10 epochs. We found that during the process, the loss on training set and the validation set has gone down steadily and the loss on the mIoU score has gone up steadily. Also, the model performance is within our expectations. Details of the validation samples are shown in the Figure 2.

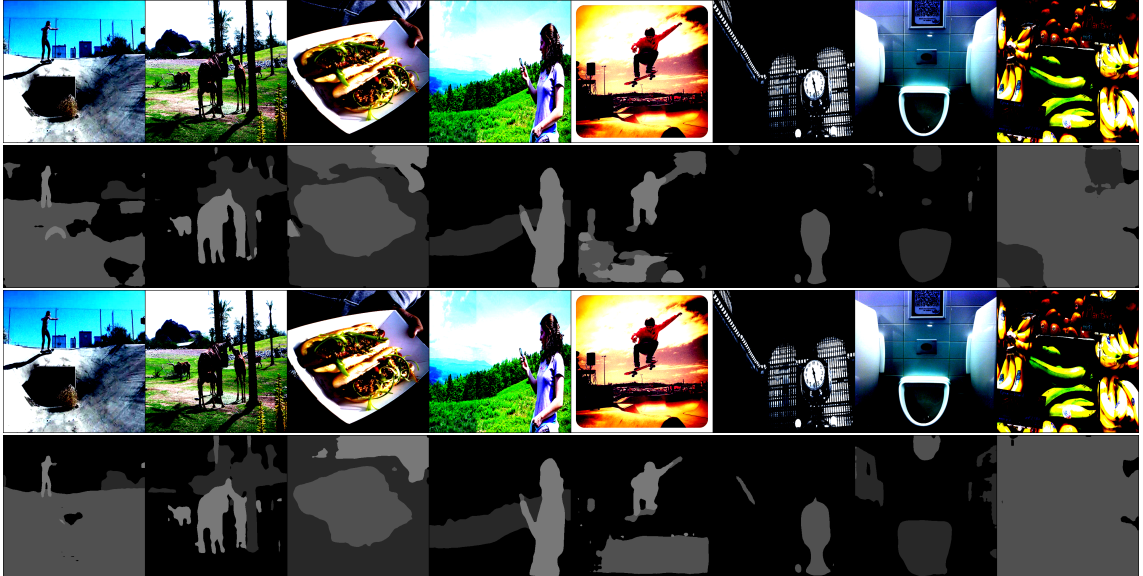


Figure 2: Segmentation Result Before and After 10 epochs

In the full scale training, we set the image number to be 9600, with a batch size of 24, learning rate $1e-4$, 50 epochs. During the training, the loss on training set decreased steadily and converges near epoch 45. The loss on validation set decreased and reached minimum at epoch 42. The mIoU score increases steadily and converges to 0.64 at epoch 50. Details of the validation samples are shown in the Figure 3.

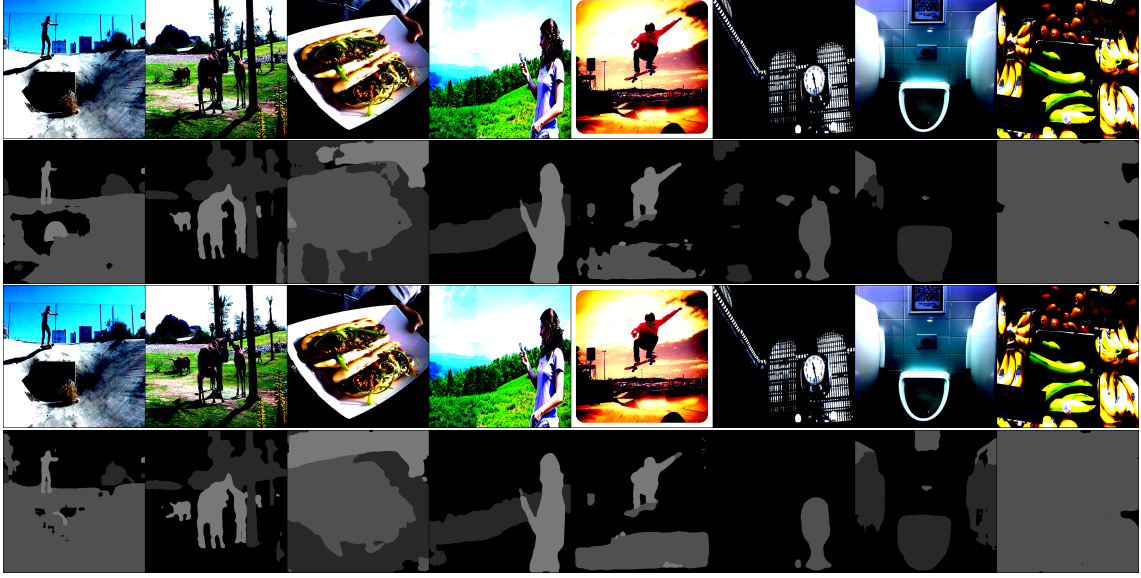


Figure 3: Segmentation Result Before and After 50 epochs

The training logs of the two stages of training could be found in Appendix B.

Evaluation

In this section, we will apply our model on test images and show the segmentation results. During the training, we mandatorily saved the models after each five steps for reference. When the model outperforms the current best model, we will save the current model as the best model. After the training session has ended, we use part of the adjusted COCO-Stuff dataset and randomly select 8 images in the dataset to perform the segmentation, results are shown in Figure 4 and Figure 5. We can see that the segmentation and semantic understanding of momentum is more accurate when training time increases. The model achieved an average inference time of 58.21 FPS per image on average within 1000 samples.

Limitations and Future Works

During our training, we tried to apply the full COCO-Stuff dataset on the task. But, because of the data distribution and the overfitting problem, the performance on the validation set dropped from epoch 5. Also, the performance is worse than the performance of the 9600 image dataset. We think this scenario may also be related to the loss function design. We would like to adjust the α and β parameter dynamically in future trainings if we can obtain more computation resources. The network have a space for improvement if we could utilize video frame datasets and strengthen the time-aware ability of the model. The approaches may include adding a 3dConv layer on the network and obtain ViPeD dataset.

Presentation Video and Ending

Thanks to prof. Hongsheng Li and his Ph.D students for supervising this project. Thanks the graders for your attention and I am also very pleased to run the whole deep learning pipeline by myself for the first time. Please refer to <https://youtu.be/oluNSNQs7zY> for the detailed video presentation on the subject.



Figure 4: Best Model Performance After Epoch 10



Figure 5: Best Model Performance After Epoch 50

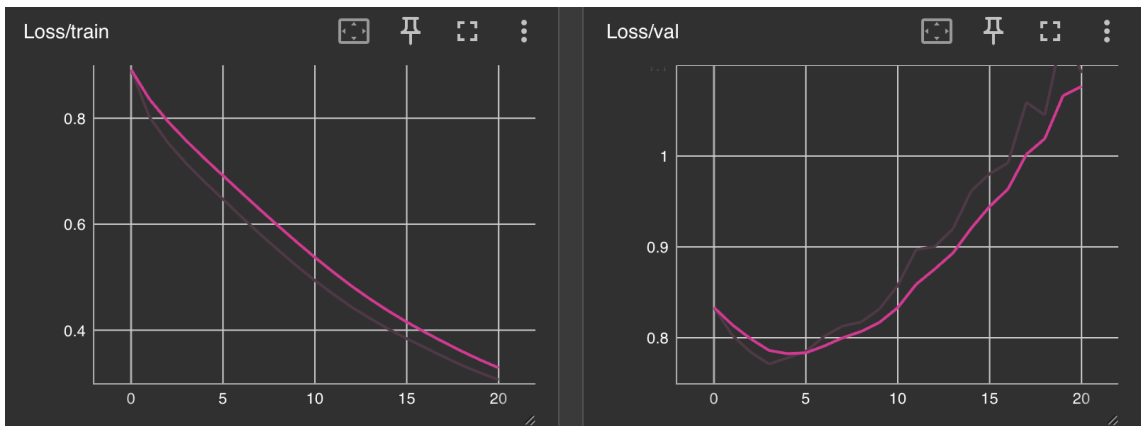


Figure 6: Train Loss and Validation Loss of Full Dataset COCO-Stuff_{Momentum}

References

- [1] Holger Caesar, Jasper Uijlings, and Vittorio Ferrari. Coco-stuff: Thing and stuff classes in context, 2018.
- [2] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. Rethinking atrous convolution for semantic image segmentation, 2017.
- [3] OpenAI. Gpt-4o system card, 2024.
- [4] Hongxing Peng, Chao Xue, Yuanyuan Shao, Keyin Chen, Juntao Xiong, Zhihua Xie, and Liuhong Zhang. Semantic segmentation of litchi branches using deeplabv3+ model. *IEEE Access*, 8:164546–164555, 2020.
- [5] Xiyang Wu, Ruiqi Xian, Tianrui Guan, Jing Liang, Souradip Chakraborty, Fuxiao Liu, Brian M. Sadler, Dinesh Manocha, and Amrit Bedi. On the safety concerns of deploying LLMs/VLMs in robotics: Highlighting the risks and vulnerabilities. In *First Vision and Language for Autonomous Driving and Robotics Workshop*, 2024.
- [6] Borong Zhang, Yuhao Zhang, Jiaming Ji, Yingshan Lei, Josef Dai, Yuanpei Chen, and Yaodong Yang. Safevla: Towards safety alignment of vision-language-action model via safe reinforcement learning, 2025.

A COCO-Stuff-Momentum Mapping

Mobility 0			Mobility 1			Mobility 2			Mobility 3		
0	10	traffic light	1	62	chair	2	26	hat	3	0	unlabeled
0	11	fire hydrant	1	63	couch	2	27	backpack	3	1	person
0	12	street sign	1	64	potted plant	2	28	umbrella	3	2	bicycle
0	13	stop sign	1	65	bed	2	29	shoe	3	3	car
0	14	parking meter	1	66	mirror	2	30	eye glasses	3	4	motorcycle
0	15	bench	1	67	dining table	2	31	handbag	3	5	airplane
0	95	bridge	1	68	window	2	32	tie	3	6	bus
0	96	building-other	1	69	desk	2	33	suitcase	3	7	train
0	97	bush	1	70	toilet	2	34	frisbee	3	8	truck
0	98	cabinet	1	71	door	2	35	skis	3	9	boat
0	99	cage	1	72	tv	2	36	snowboard	3	16	bird
0	100	cardboard	1	73	laptop	2	37	sports ball	3	17	cat
0	101	carpet	1	74	mouse	2	38	kite	3	18	dog
0	102	ceiling-other	1	75	remote	2	39	baseball bat	3	19	horse
0	103	ceiling-tile	1	76	keyboard	2	40	baseball glove	3	20	sheep
0	104	cloth	1	77	cell phone	2	41	skateboard	3	21	cow
0	105	clothes	1	78	microwave	2	42	surfboard	3	22	elephant
0	106	clouds	1	79	oven	2	43	tennis racket	3	23	bear
0	107	counter	1	80	toaster	2	44	bottle	3	24	zebra
0	108	cupboard	1	81	sink	2	45	plate	3	25	giraffe
0	109	curtain	1	82	refrigerator	2	46	wine glass			
0	110	desk-stuff	1	83	blender	2	47	cup			
0	111	dirt	1	163	straw	2	48	fork			
0	112	door-stuff	1	164	structural-other	2	49	knife			
0	113	fence	1	165	table	2	50	spoon			
0	114	floor-marble	1	166	tent	2	51	bowl			
0	115	floor-other	1	167	textile-other	2	52	banana			
0	116	floor-stone	1	168	towel	2	53	apple			
0	117	floor-tile				2	54	sandwich			
0	118	floor-wood				2	55	orange			
0	119	flower				2	56	broccoli			
0	120	fog				2	57	carrot			
0	121	food-other				2	58	hot dog			
0	122	fruit				2	59	pizza			
0	123	furniture-other				2	60	donut			
0	124	grass				2	61	cake			
0	125	gravel				2	84	book			
0	126	ground-other				2	85	clock			
0	127	hill				2	86	vase			
0	128	house				2	87	scissors			
0	132	metal				2	88	teddy bear			
0	133	mirror-stuff				2	89	hair drier			
0	134	moss				2	90	toothbrush			
0	135	mountain				2	91	hair brush			
0	136	mud				2	92	banner			
0	140	pavement				2	93	blanket			
0	144	platform				2	94	branch			
0	145	playingfield				2	129	leaves			
0	146	railing				2	130	light			
0	147	railroad				2	131	mat			
0	148	river				2	137	napkin			
0	149	road				2	138	net			
0	150	rock				2	139	paper			
0	151	roof				2	141	pillow			
0	152	rug				2	142	plant-other			
0	155	sea				2	143	plastic			
0	156	shelf				2	153	salad			
0	157	sky-other				2	154	sand			

B The Training Log of Stage 2

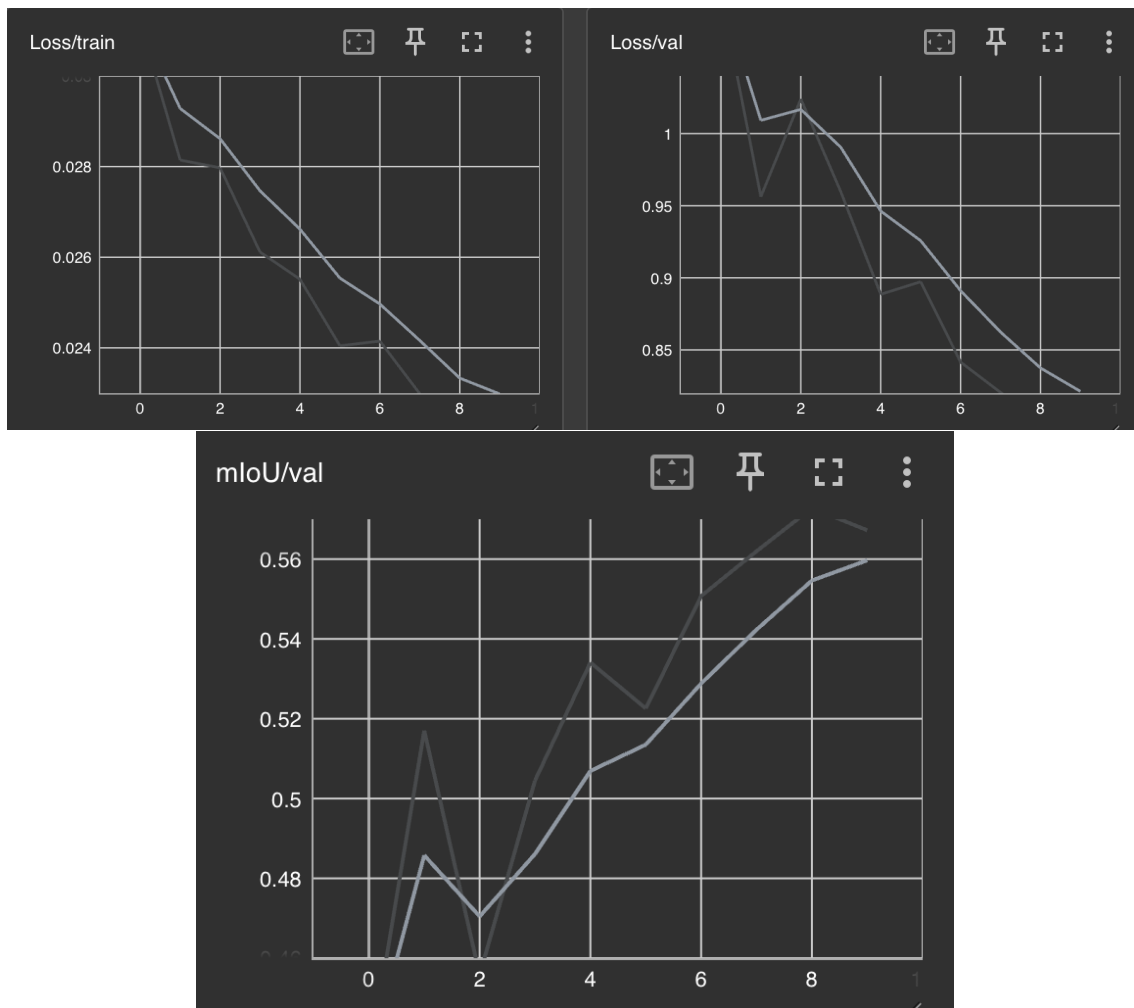


Figure 7: Training Loss, Validation Loss and IoU Score Changes On 10 Epoches

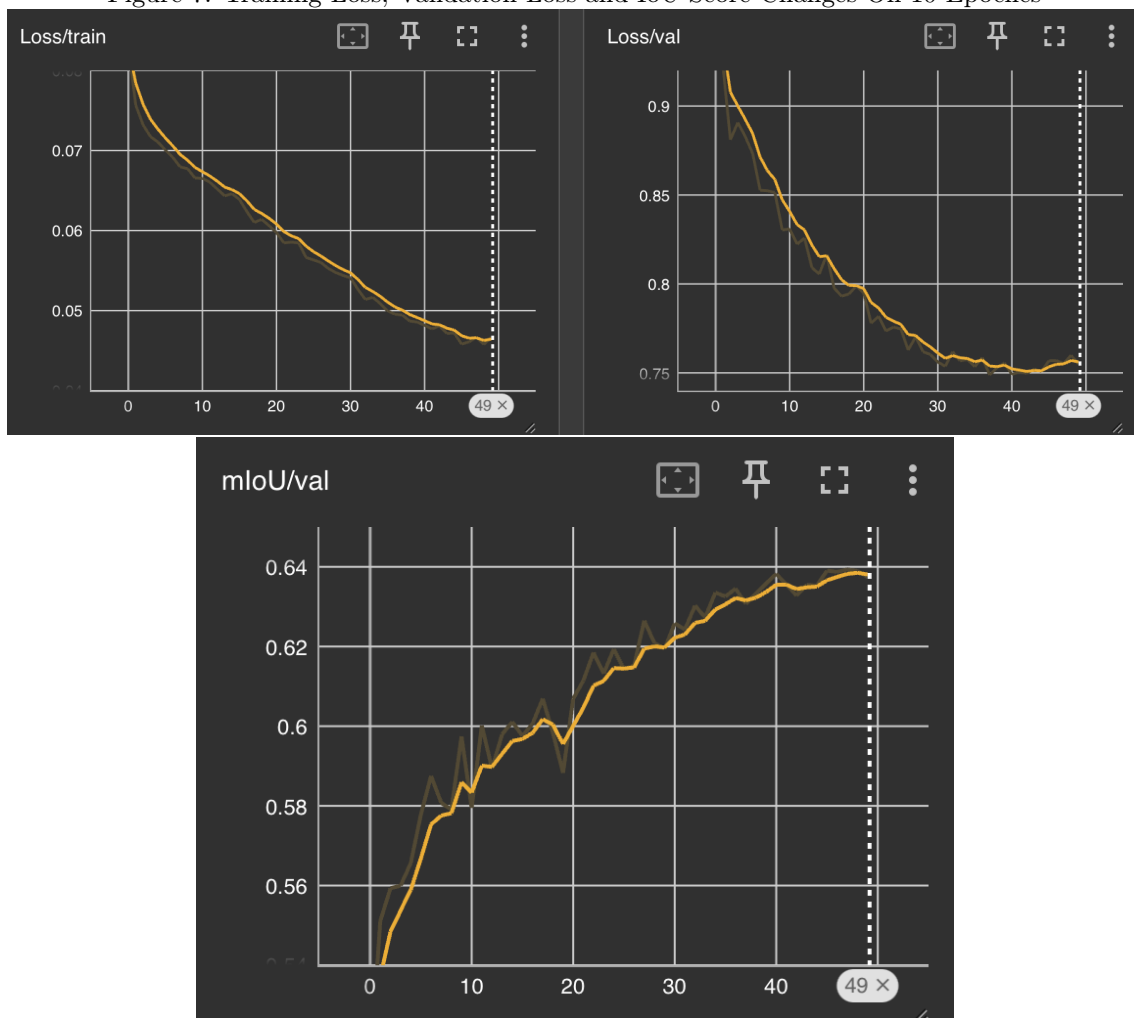


Figure 8: Training Loss, Validation Loss and IoU Score Changes On 50 Epoches